MLOps Batch Signal Job

A minimal MLOps-style batch job that computes a rolling-mean-based binary trading signal on OHLCV data.
Demonstrates **reproducibility**, **observability**, and **deployment readiness**.

Project structure

```
mlops-task/
├── run.py             # Main processing script
├── config.yaml        # Job configuration
├── data.csv           # Input OHLCV dataset (10 000 rows)
├── requirements.txt   # Python dependencies
├── Dockerfile         # Container definition
├── metrics.json       # Sample output from a successful run
├── run.log            # Sample log from a successful run
└── README.md          # This file
```

Local run

Prerequisites

- Python 3.9+
- pip

Install dependencies

```
pip install -r requirements.txt
```

Run the job

```
python run.py \
  --input    data.csv \
  --config   config.yaml \
  --output   metrics.json \
```

```
  --log-file run.log
```

All four arguments are required; no paths are hard-coded.

Docker build & run

```
# Build the image
docker build -t mlops-task .

# Run the container (uses defaults baked into CMD)
docker run --rm mlops-task
```

The container:

- bundles `data.csv` and `config.yaml`
- writes `metrics.json` and `run.log` inside the container
- prints the final metrics JSON to **stdout**
- exits **0** on success, **non-zero** on failure

Inspect output files from a named container

```
docker run --name mlops-run mlops-task
docker cp mlops-run:/app/metrics.json .
docker cp mlops-run:/app/run.log .
docker rm mlops-run
```

Configuration (`config.yaml`)

Key	Type	Description
`seed`	int	NumPy random seed (reproducibility)
`window`	int	Rolling mean window size
`version`	string	Job version label in output JSON

Signal logic

1. Compute `rolling_mean = close.rolling(window).mean()`
2. For each row where `rolling_mean` is not NaN:
   `signal = 1 if close > rolling_mean else 0`
3. First `window - 1` rows are excluded from signal computation (NaN warm-up period).

Example `metrics.json`

```
{
  "version": "v1",
  "rows_processed": 9996,
  "metric": "signal_rate",
  "value": 0.4991,
  "latency_ms": 28,
  "seed": 42,
  "status": "success"
}
```

Error output format

```
{
  "version": "v1",
  "status": "error",
  "error_message": "Description of what went wrong"
}
```

Metrics are **always** written — even on failure.

Validation & error handling

The script cleanly handles:

- Missing input file or config file
- Invalid / non-parseable CSV
- Empty CSV

- Missing `close` column
- Non-numeric values in `close` (dropped with a warning)
- Missing or wrong-type config keys

Rubric coverage

Criterion	Implementation
Correctness	Deterministic via `numpy.random.seed(seed)` + YAML config
Metrics format	Exact keys per spec; written in both success & error cases
Dockerization	`python:3.9-slim` ; no hard-coded paths; CMD uses CLI flags
Code quality	Modular functions, typed, full validation, clean error flow
Observability	Structured timestamped logs covering every processing step